

```

from __future__ import annotations

from collections import defaultdict, deque
from datetime import timedelta
from math import asin, cos, radians, sin, sqrt
from pathlib import Path

import pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parents[1]
INPUT_PATH = PROJECT_ROOT / "data" / "synthetic_access_logs.csv"
OUTPUT_PATH = PROJECT_ROOT / "data" / "processed_access_logs.csv"
BASELINE_PATH = PROJECT_ROOT / "models" / "entity_baseline_profiles.csv"
DRIFT_CONFIRMATIONS = 3

def haversine_km(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    """Return geographical distance between two coordinates in kilometres."""
    earth_radius_km = 6371.0

    delta_lat = radians(lat2 - lat1)
    delta_lon = radians(lon2 - lon1)

    a = (
        sin(delta_lat / 2) ** 2
        + cos(radians(lat1))
        * cos(radians(lat2))
        * sin(delta_lon / 2) ** 2
    )
    return 2 * earth_radius_km * asin(sqrt(a))

def _make_profile(group: pd.DataFrame) -> dict:
    """Build a reusable behavioural profile from an already trusted window."""
    bytes_std = group["bytes_transferred"].std(ddof=0)
    if pd.isna(bytes_std) or bytes_std < 1:
        bytes_std = 1.0
    return {
        "event_count": len(group),
        "median_hour": float(group["hour"].median()),
        "hour_std": max(float(group["hour"].std(ddof=0) or 1), 1.0),
        "median_bytes": float(group["bytes_transferred"].median()),
        "bytes_std": float(bytes_std),
        "locations": set(group["geo_location"]),
        "resources": set(group["resource_accessed"]),
        "fingerprints": set(group["device_fingerprint"]),
        "source_ips": set(group["source_ip"]),
        "command_sequences": set(group["command_sequence"]),
    }

def build_baselines(data: pd.DataFrame) -> tuple[dict, dict, pd.Timestamp]:
    """
    Learn per-entity normal behaviour using the first 14 days of history.
    Later events are scored against this historical baseline.
    """
    baseline_cutoff = data["timestamp"].min() + pd.Timedelta(days=14)
    historical_data = data[data["timestamp"] < baseline_cutoff].copy()

    baselines = {}
    population_baselines = {}
    export_rows = []

    for entity_id, group in historical_data.groupby("entity_id"):
        profile = _make_profile(group)

        baselines[entity_id] = profile

        export_rows.append(
            {
                "entity_id": entity_id,
                "baseline_event_count": profile["event_count"],
                "baseline_median_hour": round(profile["median_hour"], 2),
                "baseline_hour_std": round(profile["hour_std"], 2),
                "baseline_median_bytes": round(profile["median_bytes"], 2),
                "known_locations": " | ".join(sorted(profile["locations"])),
                "known_resources": " | ".join(sorted(profile["resources"])),
                "known_fingerprints": " | ".join(
                    sorted(profile["fingerprints"])
                ),
            },
        )

    for entity_type, group in historical_data.groupby("entity_type"):
        population_baselines[entity_type] = _make_profile(group)

    pd.DataFrame(export_rows).to_csv(BASELINE_PATH, index=False)
    return baselines, population_baselines, baseline_cutoff

def add_baseline_features(
    data: pd.DataFrame,
    baselines: dict,
    population_baselines: dict,
) -> pd.DataFrame:
    """Compare events with an entity baseline, adapting confirmed safe drift.

    New entities are scored against their entity-type population profile instead
    of being treated as automatically benign. For established entities, a new
    resource/command is accepted only after repeated safe observations, so a
    legitimate change is not flagged forever while obvious attack context is
    never used to update the profile.
    """
    feature_rows = []
    adaptive_profiles = {
        entity_id: {
            **profile,
            "locations": set(profile["locations"]),
            "resources": set(profile["resources"]),
            "fingerprints": set(profile["fingerprints"]),
            "source_ips": set(profile["source_ips"]),
            "command_sequences": set(profile["command_sequences"]),
        }
        for entity_id, profile in baselines.items()
    }
    safe_novelty_counts = defaultdict(int)
    safe_novelty_values = defaultdict(list)

    for row in data.itertuples(index=False):
        baseline = adaptive_profiles.get(row.entity_id)
        baseline_source = "entity"

```

```

# Cold starts use a population baseline for the same entity type.
if baseline is None:
    baseline = population_baselines.get(row.entity_type)
    baseline_source = "population"

if baseline is None:
    feature_rows.append(
        {
            "baseline_event_count": 0,
            "hour_deviation": 0.0,
            "bytes_zscore": 0.0,
            "new_location": 0,
            "new_resource": 0,
            "new_device": 0,
            "new_source_ip": 0,
            "new_command_sequence": 0,
            "baseline_source": "none",
            "concept_drift_adapted": 0,
        }
    )
    continue

values = {
    "baseline_event_count": baseline["event_count"],
    "hour_deviation": abs(row.hour - baseline["median_hour"]),
    "bytes_zscore": abs(row.bytes_transferred - baseline["median_bytes"])
    / baseline["bytes_std"],
    "new_location": int(row.geo_location not in baseline["locations"]),
    "new_resource": int(row.resource_accessed not in baseline["resources"]),
    "new_device": int(row.device_fingerprint not in baseline["fingerprints"]),
    "new_source_ip": int(row.source_ip not in baseline["source_ips"]),
    "new_command_sequence": int(
        row.command_sequence not in baseline["command_sequences"]
    ),
    "baseline_source": baseline_source,
    "concept_drift_adapted": 0,
}
feature_rows.append(values)

# Only a familiar, successful context can establish benign drift.
safe_context = (
    baseline_source == "entity"
    and row.auth_result == "success"
    and values["new_location"] == 0
    and values["new_device"] == 0
    and values["new_source_ip"] == 0
    and 7 <= row.hour <= 20
)

if safe_context and (values["new_resource"] or values["new_command_sequence"]):
    safe_novelty_counts[row.entity_id] += 1
    safe_novelty_values[row.entity_id].append(
        (row.resource_accessed, row.command_sequence)
    )
    if safe_novelty_counts[row.entity_id] >= DRIFT_CONFIRMATIONS:
        for resource, command in safe_novelty_values.pop(row.entity_id):
            baseline["resources"].add(resource)
            baseline["command_sequences"].add(command)
        safe_novelty_counts[row.entity_id] = 0
        feature_rows[-1]["concept_drift_adapted"] = 1

return pd.concat(
    [data.reset_index(drop=True), pd.DataFrame(feature_rows)],
    axis=1,
)

```

```

def add_temporal_features(data: pd.DataFrame) -> pd.DataFrame:
    """
    Add rolling-window features and geo-velocity.
    These reveal brute force, credential stuffing, and impossible travel.
    """
    data = data.sort_values("timestamp").reset_index(drop=True).copy()

    ip_windows = defaultdict(deque)
    entity_windows = defaultdict(deque)
    entity_history_count = defaultdict(int)
    previous_event = {}

    ip_attempts_1h = []
    ip_unique_entities_1h = []
    entity_events_24h = []
    history_count = []
    distance_from_previous = []
    travel_speed_kmh = []
    impossible_travel = []

    for row in data.iteruples(index=False):
        timestamp = row.timestamp
        entity_id = row.entity_id
        source_ip = row.source_ip

        # Events from the same IP during the last hour.
        ip_queue = ip_windows[source_ip]
        while ip_queue and timestamp - ip_queue[0][0] > timedelta(hours=1):
            ip_queue.popleft()

        recent_entities = {item[1] for item in ip_queue}
        ip_attempts_1h.append(len(ip_queue) + 1)
        ip_unique_entities_1h.append(len(recent_entities | {entity_id}))
        ip_queue.append((timestamp, entity_id))

        # Recent activity and history count for each entity.
        entity_queue = entity_windows[entity_id]
        while (
            entity_queue
            and timestamp - entity_queue[0] > timedelta(hours=24)
        ):
            entity_queue.popleft()

        entity_events_24h.append(len(entity_queue) + 1)
        history_count.append(entity_history_count[entity_id])
        entity_queue.append(timestamp)
        entity_history_count[entity_id] += 1

        # Travel velocity from this entity's previous event.
        previous = previous_event.get(entity_id)
        if previous is None:
            distance_from_previous.append(0.0)
            travel_speed_kmh.append(0.0)
            impossible_travel.append(0)
        else:
            previous_time, previous_lat, previous_lon = previous
            time_gap_hours = (

```

```

        timestamp - previous_time
    ).total_seconds() / 3600

    distance = haversine_km(
        previous_lat,
        previous_lon,
        row.geo_lat,
        row.geo_lon,
    )

    if time_gap_hours <= 0:
        speed = 0.0
    else:
        speed = distance / time_gap_hours

    distance_from_previous.append(round(distance, 2))
    travel_speed_kmh.append(round(speed, 2))
    impossible_travel.append(
        int(distance > 500 and 0 < time_gap_hours < 2)
    )

    previous_event[entity_id] = (
        timestamp,
        row.geo_lat,
        row.geo_lon,
    )

data["source_ip_attempts_1h"] = ip_attempts_1h
data["source_ip_unique_entities_1h"] = ip_unique_entities_1h
data["entity_events_24h"] = entity_events_24h
data["entity_history_count"] = history_count
data["cold_start_entity"] = (data["entity_history_count"] < 10).astype(int)
data["distance_from_previous_km"] = distance_from_previous
data["travel_speed_kmh"] = travel_speed_kmh
data["impossible_travel_flag"] = impossible_travel

return data

def main() -> None:
    if not INPUT_PATH.exists():
        raise FileNotFoundError(
            "Synthetic logs are missing. Run: python src\\generate_data.py"
        )

    data = pd.read_csv(INPUT_PATH, parse_dates=["timestamp"])
    data = data.sort_values("timestamp").reset_index(drop=True)

    # Simple time-derived behaviour signals.
    data["hour"] = data["timestamp"].dt.hour
    data["day_of_week"] = data["timestamp"].dt.dayofweek
    data["is_weekend"] = (data["day_of_week"] >= 5).astype(int)
    data["is_off_hours"] = (
        (data["hour"] < 7) | (data["hour"] > 20)
    ).astype(int)
    data["failed_auth"] = (data["auth_result"] == "failure").astype(int)

    baselines, population_baselines, cutoff = build_baselines(data)
    data = add_baseline_features(data, baselines, population_baselines)
    data = add_temporal_features(data)

    data["bytes_zscore"] = data["bytes_zscore"].clip(upper=25)
    data["travel_speed_kmh"] = data["travel_speed_kmh"].clip(upper=50_000)

    data.to_csv(OUTPUT_PATH, index=False)

    print(f"Baseline period ends: {cutoff.date()}")
    print(f"Created processed feature dataset: {OUTPUT_PATH}")
    print(f"Created entity baseline profiles: {BASELINE_PATH}")
    print("\nFeature checks:")
    print(
        data[
            [
                "new_location",
                "new_resource",
                "new_device",
                "new_source_ip",
                "impossible_travel_flag",
                "is_off_hours",
            ]
        ]
        .sum()
        .to_string()
    )

if __name__ == "__main__":
    main()

```